Maruf Sir - Part A

Compiler Design | Exam suggestions

Highest emphasis: Sir singled out **syllabus point 5**, especially recursive-descent parsing, left recursion and its elimination, as the source of a large part of the **mandatory question**. Read the slides carefully.

1. Language processing system and compiler

- ☐ Define a **language processing system** and draw its diagram. Explain the roles of the preprocessor, compiler, assembler, linker and loader.
- ☐ Define a **compiler** and explain its role, structure and phases of compilation. Give particular attention to **lexical analysis and syntax analysis**.

2. Lexical analysis and token recognition

- ☐ Explain **lexical analysis** and the lexical analyzer/scanner's work and role. Understand how source characters become tokens for the parser.
- ☐ Define and distinguish **token, lexeme and pattern**, using examples.
- ☐ Given a code block, **identify and count tokens**, including valid tokens and invalid token-like sequences under the stated language rules. Show a line-by-line breakdown.
- ☐ **Very important:** recognize tokens using **regular expressions (RE), NFA and DFA**. Practise keywords and identifiers in particular, including their patterns and automata.

3. Input buffering and transition diagrams

- ☐ **Very important:** explain **input buffering**. Review the slide diagrams, begin/lookahead pointers, the two-buffer scheme and sentinels.
- ☐ Draw detailed **transition diagrams** for **keywords/reserved words, identifiers, arithmetic operators, relational operators and logical operators**. The syllabus also includes other common tokens.
- ☐ Show states, labelled transitions, the start state and accepting states. Include lookahead/retraction where the slide example requires it.

Sir's warning about diagram answers

In the midterm, students lost marks by drawing the **wrong operator category**, covering only a small subset, or giving an overly brief answer. Draw the category actually requested, cover its relevant operators, and match the detail to the marks.

Maruf Sir - Part A

Compiler Design | Exam suggestions

4. Grammar and derivations

- ☐ Define **grammar and context-free grammar (CFG)**. Identify terminals, non-terminals, production rules and the start symbol.
- ☐ Given a grammar and target string, perform **leftmost and rightmost derivations**. Show the intermediate steps clearly.

5. Syntax analysis and parsing - highest priority

- ☐ Explain **syntax analysis**, the syntax analyzer's role/responsibilities, the phases of syntax analysis and the role of parsing.
- ☐ Distinguish **top-down and bottom-up parsing**. Prepare **recursive-descent parsing**: one function per non-terminal, production choices and matching input.
- ☐ Define and **detect left recursion**. Distinguish **direct and indirect left recursion**.
- ☐ Explain why left recursion causes problems for a recursive-descent parser: a function calls itself again **before consuming input**, causing infinite recursion and possible stack overflow.
- ☐ **Practise eliminating left recursion** and transforming a left-recursive grammar into an **equivalent non-left-recursive grammar**. Show each transformation step and explain the resulting grammar.

  Handout practice example:
  Before: S -> Sa | b
  After: S -> bS' and S' -> aS' | epsilon
  Explain why both generate b followed by zero or more a symbols, and why the new form avoids the original recursive-descent problem.

6. Lexical errors versus syntax errors

- ☐ Define both error types, their **common causes and detection**. Given a program containing both, list and classify each error correctly.
- ☐ Distinguish an **invalid token** from a **grammatically invalid arrangement of valid tokens**. A missing semicolon is a syntax error. Apply the stated language rules when judging token validity.

Additional practice flagged in the handnote

Prepare **parse-tree construction** and **ambiguity**: demonstrate ambiguity with two distinct parse trees or two distinct leftmost/rightmost derivations for the same string. Review **precedence, associativity and removing ambiguity** as handnote self-study topics. These were **not explicitly singled out in the recorded final suggestion**.

Study order (recommended): Point 5 first; then RE/NFA/DFA, buffering and transition diagrams; token counting and error classification; grammar/derivations; compiler fundamentals. This order reflects the emphasis, not confirmed mark distribution.